What is a Phantom deadlock?

In DBMS, we all must have heard the term deadlock. A deadlock can be defined as a situation where processes wait indefinitely for some resources that are already acquired by some processes. You can learn more about the deadlock in DBMS here .

In this blog, we will learn about a specific deadlock, i.e., Phantom deadlock. We will understand how Phantom deadlock occurs in DBMS. So, let us start our learning.

Phantom Deadlock

Phantom Deadlock is a deadlock that occurs in a Distributed DBMS due to communication delays between different processes and leads to unnecessary process abortions. In fact, it is not really a deadlock.

In Distributed DBMS, it is quite difficult to detect deadlock because several processes may release some resources, or request for some other resources at the same time. The Distributed DBMS can use any deadlock detection algorithm to detect the deadlock. But, the deadlock is detected in a centralized manner. Whenever a process releases some resources or requests for some other resources, it sends a release or request message to the centralized system respectively.

In some cases, the resource release messages arrive late than the resource request messages for some specific resources. The deadlock detection algorithm treats the situation as a deadlock and may abort some processes to break the deadlock. But in actual, no deadlocks were present, the deadlock detection algorithms detected the deadlock based on outdated messages from the processes due to communication delays. Such a deadlock is called `Phantom Deadlock` .

Example

Now, let us understand how phantom deadlock occurs with the help of an example.

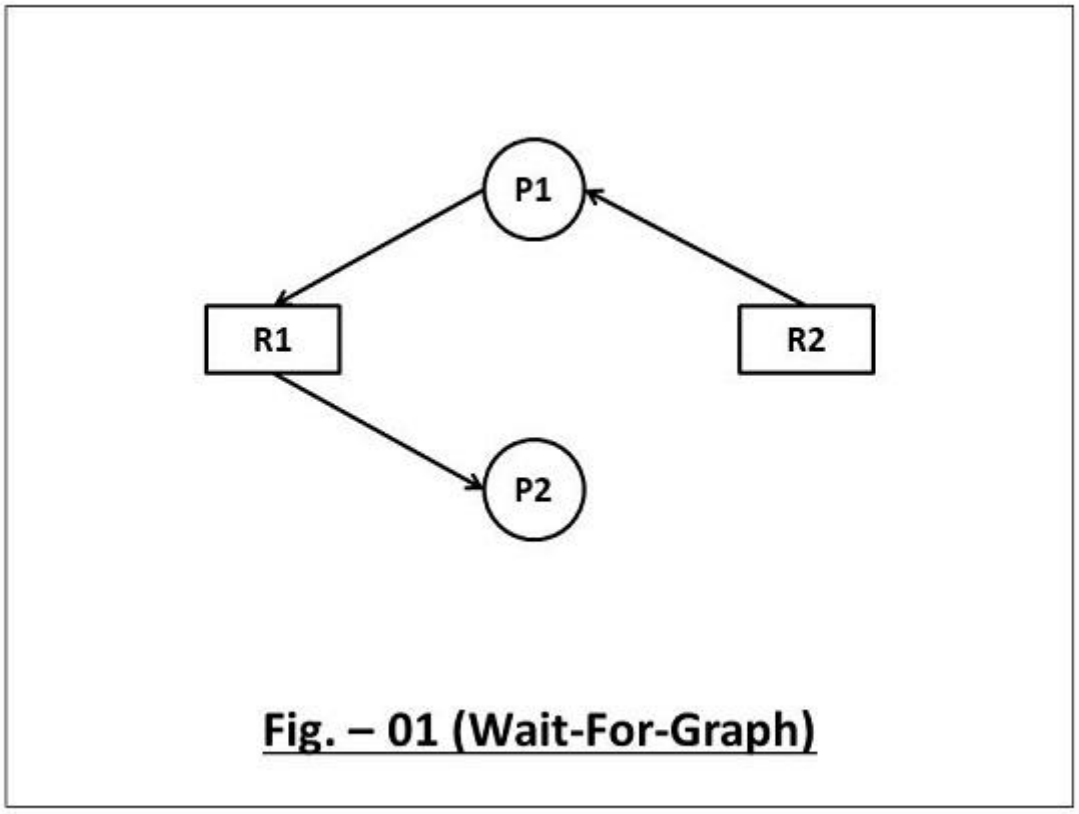

Fig. – 01 (Wait-For-Graph)

The above diagram is a wait-for-graph for two processes - `P1 and P2` respectively. You can also see two resources `R1 and R2` . Process P1 holds the resource R2, and request for the resource R1. On the other hand, Process P2 holds the resource R1. Currently, there is no deadlock in the system.

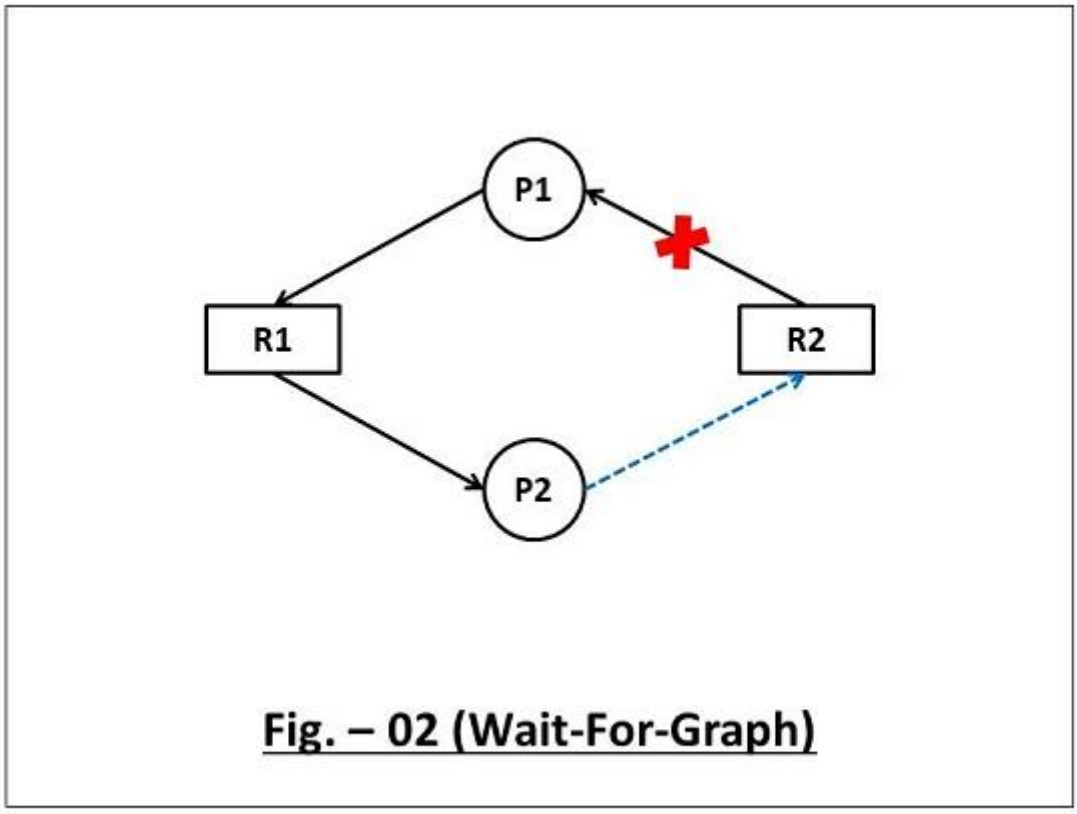

Fig. – 02 (Wait-For-Graph)

Now in the above diagram, you can see, process P1 releases resource R2 and process P2 has requested resource R2. In practical scenarios, P1 will send a release message, and P2 will send a request message for resource R2 to the centralized system.

Now, assume the case where the resource request message has arrived for resource R2 first. On the other hand, the release message has not yet arrived for the resource R2. In such a case, the deadlock detection algorithm will treat it as a deadlock and will unnecessarily abort some processes to break this deadlock. But in actual, there is no such deadlock.

This is all about the Phantom deadlock in DBMS. Hope you learned something new today. That's it for this blog.